Phase 1 Report

Preparing the data for a study of RAG techniques in image captioning

Group

Names: Arslane Aicha Razen, Aya louetri, Zerd lina assia, Hiba hassani

November 2026

Summary

This report covers the phase of our project. The main goal of the project is to compare RAG techniques, which stands for Retrieval-Augmented Generation, when it comes to writing captions for images. Before we can start comparing anything every team must have the clean and consistent data. Phase 1 is focused on building that data.

We created a knowledge base that includes 10,000 COCO training images and their 50,000 corresponding captions. We also added 100 validation images and two test sets, one with 100 images and one with 1,000 images. No image appears in than one set. We carefully checked all the images for missing files, broken files and duplicate files. We found none. Then we used a detector called DETR to identify objects inside each image. We analyzed the data before. After preprocessing to understand how it changed. After that we turned all the images and captions into CLIP vectors. Finally we built two FAISS indexes, one for images and one for captions, so that similar items can be found in a few milliseconds.

All the steps are written as functions inside a notebook. We used a fixed seed to make sure the results are reproducible. We also saved all the results, on Google Drive so that anyone can repeat the work exactly as we did.

1. Introduction

1.1 Why this phase exists

Modern captioning models write fluent sentences, but they sometimes forget objects, invent objects that are not there, or are vague about colours and positions. RAG tries to fix this by letting the model look at similar images and their captions before it writes. To compare RAG techniques fairly, all groups must use the same images, the same captions and the same search tools. Phase 1 creates them.

1.2 What this phase delivers

- A cleaned dataset divided into three separate sets: knowledge base, validation, and test.
- Clearly documented data splits, with verification that test images are not included in the knowledge base.
- Statistics and visualizations showing the dataset's characteristics before and after preprocessing.
- Objects detected in the images, extracted and stored as metadata.
- Image and caption embeddings generated and saved, along with FAISS indexes prepared for efficient retrieval.

2. Data and splits

2.1 The three sets

All images come from MS COCO. We keep three kinds of images, and each has its own job. The knowledge base is the collection we search in later. The validation set is used to make choices and tune settings during development. The test set is used only at the end, to measure the final quality.

Set	Images	Source	Captions	Job
Knowledge base	10,000	train2017	50,000	Search collection
Validation	100	train2017, different from the knowledge base	500	Tuning and choices
Test A	100	val2017	500	First test set
Test B	1,000	val2017 (contains Test A)	5,000	Main test set

The validation images come from the same COCO split as the knowledge base but are different images. We cut both from the same shuffled list: the first 10,000 images form the knowledge base and the next 100 form the validation set. This makes an overlap impossible. Together they use 10,100 images of train2017.

2.2 Why there are two test sets

The project text mentions both 100 and 1,000 test images. We first built a test set of 100 images (Test A). When the teacher asked for 1,000 images, we extended it to Test B and kept Test A inside it. The 100 images are the first 100 images of the 1,000. This way, results can be reported on both sets and compared.

Both test sets are stratified. This means the images are chosen on purpose to cover three kinds of scenes in equal parts: simple (1 or 2 objects), medium (3 to 5 objects) and complex (6 or more objects). Test A has 34, 33 and 33 images of these kinds. Test B has 334, 333 and 333. This avoids a test set made only of easy or only of crowded scenes.

2.3 Selection rules

- Only images with exactly five captions are used.
- Test images must contain at least one object.
- The selection uses a fixed random seed (42; the 900 extra test images use seed 43), so anyone who runs the code gets exactly the same images.
- Spare images are kept in reserve. If an image has to be removed, a spare takes its place and the set keeps its size.

3. Data audit and cleaning

Before using the images we checked them. Four functions do this work. download() downloads many images at the same time. It writes each image to a temporary file first, so a stopped download never leaves a half image. inspect_image() opens each file once. It reports whether the image is missing, corrupted or fine, and computes its MD5 fingerprint, a short code that is the same for identical files. build_clean_set() uses these two to download and check a whole set. It removes missing, corrupted and duplicate images, keeps one copy of each duplicate, and replaces the removed images with spares. build_part1() runs everything for the knowledge base, the validation set and Test A. extend_test_set() then adds the 900 extra test images.

Result. No image was missing, corrupted or duplicated in the knowledge base, the validation set, Test A or the 900 new images of Test B. The fingerprint file of our first run was lost, so we computed the

fingerprints again for all 10,200 final images (this took about five minutes). The cleaning report therefore describes the final sets. It does not list the removals of the first selection.

Leakage audit. We compared every pair of sets, by image id and by fingerprint. No id and no identical picture is shared between any two sets. The code stops with an error if it ever finds one. We also checked that the five captions of every image exist and are not empty.

4. Preprocessing

4.1 Images

Every image is rotated according to its EXIF tag (the tag that says how the camera was held) and converted to RGB (three colour channels). In our data no image needed rotation and 15 of the 10,000 knowledge-base images were grayscale, so the conversion mattered for those. Resizing and normalising are done by the CLIP processor, as the CLIP model requires. The same preparation is used before object detection, so the detector and the encoder see the same picture.

4.2 Captions

Each caption is stored twice. caption_raw is the original sentence and is never changed; it is the only one used for evaluation. caption_clean is the lowercase version without punctuation; it is used only for search and embeddings. The function clean_text() makes the clean version. Keeping both avoids mixing evaluation data with search data.

5. Object extraction from the images

COCO already has human object labels, but we also wanted objects that come directly from the image. These are available for any new image, including test images, whose human labels we are not allowed to use for retrieval. We used the DETR detector (facebook/detr-resnet-50), which recognises the 80 COCO object classes. For every detected object we store its label, its confidence score and its box. The function detect_objects() runs the detector in batches of 16 on the GPU and saves its progress every 1,000 images. It processed 11,100 images (knowledge base, validation and Test B) in about 53 minutes.

5.1 Choosing the confidence threshold

A detector gives each object a score between 0 and 1. A low threshold keeps more objects but also more mistakes; a high threshold keeps fewer, more reliable objects. We tried thresholds from 0.5 to 0.9 on the 100 validation images and compared the detected classes with the human labels. At 0.5, precision was 0.882 and recall 0.958. At 0.9, precision rose to 0.975 and recall fell to 0.902. The F1 score, which balances both, was best at 0.9 (0.937), so we chose 0.9. The test images were not used for this choice.

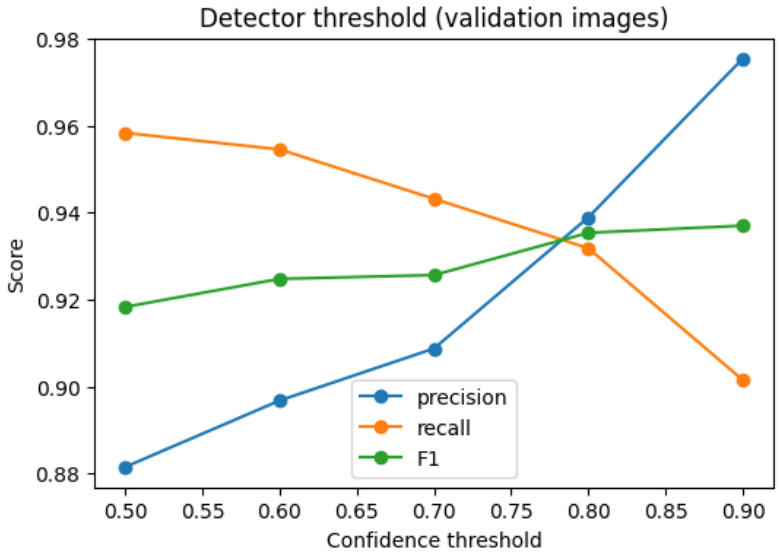


Figure 1. Precision, recall and F1 of the detector for different thresholds (validation images).

5.2 How good is the detector?

On the knowledge base, the detected classes agree well with the human labels (precision 0.950, recall 0.893, F1 0.920). On Test B they agree less (precision 0.914, recall 0.795, F1 0.850), and on Test A the results are similar (0.897, 0.813, 0.853). So the detector rarely invents classes, but it misses about one human-labelled class in five on the test images, mostly small or partly hidden objects.

5.3 Confidence of the metadata

We treat the two kinds of information differently. Data written by people (COCO captions, COCO object labels, and the colour or place words we read from the captions) gets confidence 1, because it is human-made. Objects found by the detector keep their own score and are kept only if the score is at least 0.9.

6. Data exploration

We studied the knowledge base twice: once on the raw data (before preprocessing) and once on the prepared data (after preprocessing). After preprocessing, the statistics start from the objects found by the detector.

6.1 Before preprocessing

The images are 579 by 483 pixels on average. 9,985 are RGB and 15 are grayscale. The human labels contain 80 object classes and 7.19 objects per image on average; 90 images have no labelled object. By scene type, 4,251 images are complex, 3,046 simple and 2,703 medium, so the knowledge base has many crowded scenes. Most images are about 640 pixels wide, so resizing is needed. A caption has 10.45 words on average. The raw captions use 16,255 different words, but many of them are the same word written with a capital letter or with a punctuation mark attached.

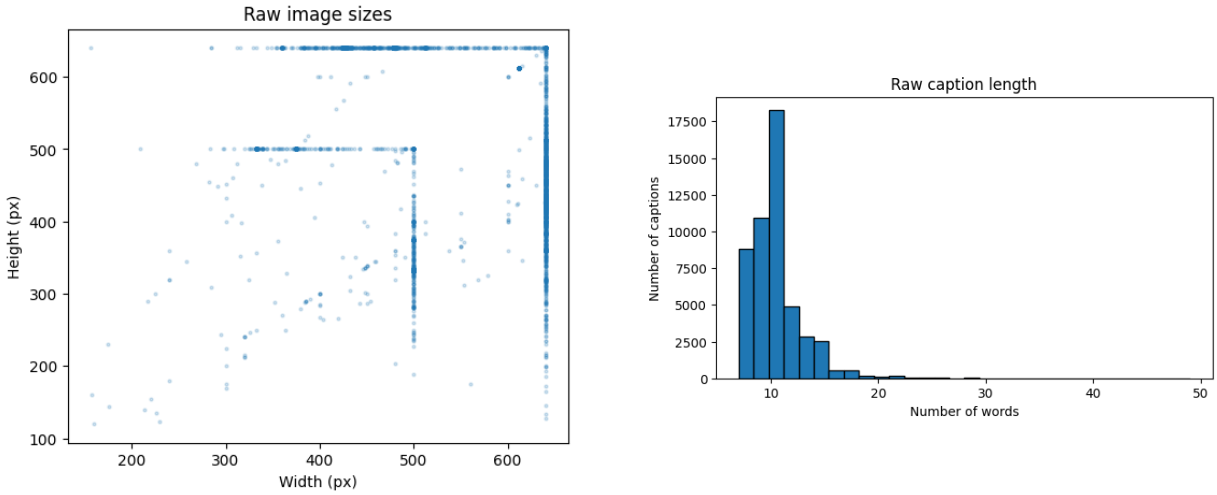


Figure 2. Raw image sizes (left) and raw caption length in words (right).

6.2 After preprocessing

The detector finds 6.15 objects per image, about one fewer than the human labels, and finds nothing in 100 images. The scene types become 3,213 simple, 2,871 medium and 3,916 complex, a similar picture. The average detection confidence of the kept objects is 0.975. After cleaning, the vocabulary falls from 16,255 to 10,080 words because equal words now match. The caption length does not change, since cleaning removes punctuation but not words.

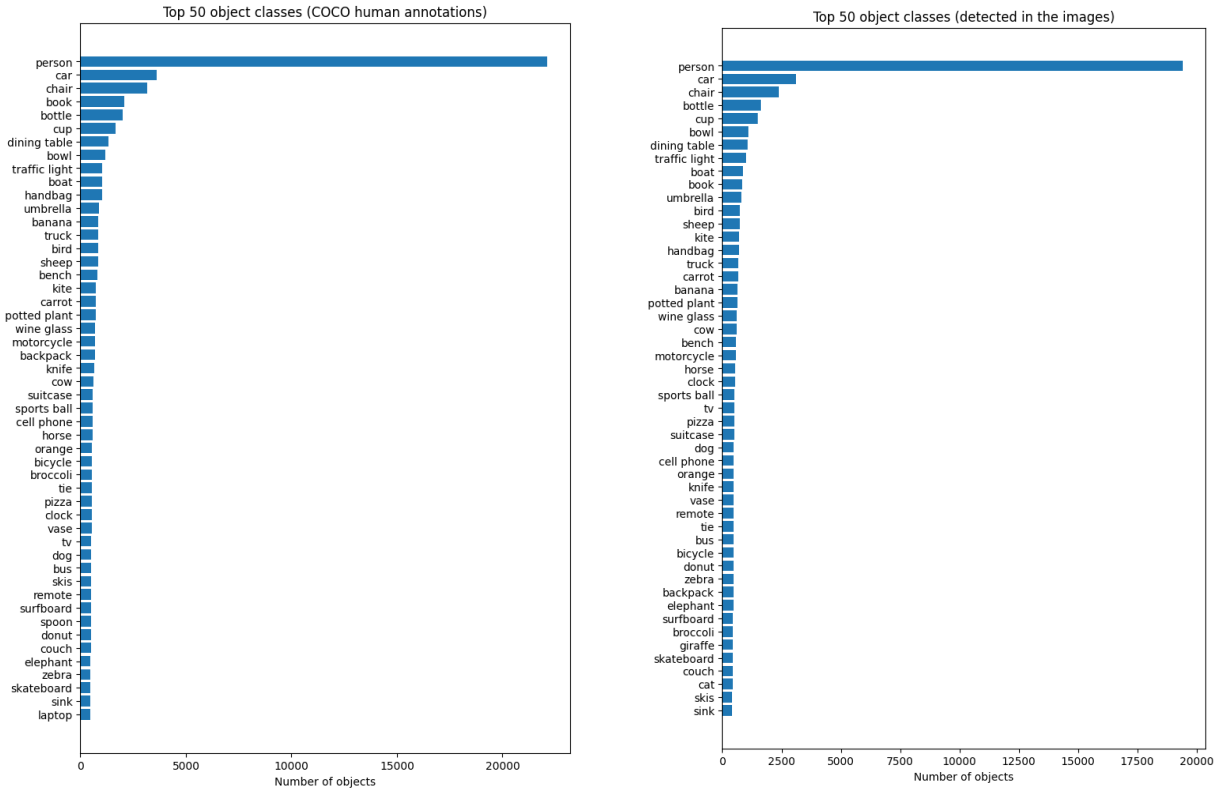


Figure 3. The 50 most frequent object classes: human COCO labels (left) and objects found by the detector (right). Person is by far the most frequent in both.

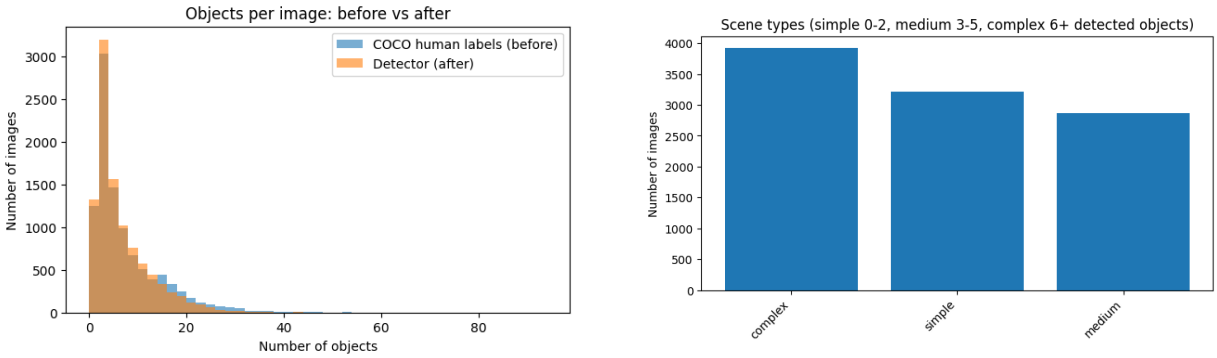


Figure 4. Objects per image, human labels against detector (left), and scene types from the detector (right).

6.3 The captions

We tagged 5,000 random captions (52,200 words) with their part of speech. Nouns are the biggest group (35.5%), followed by determiners (20.6%), prepositions (16.1%), verbs (11.7%) and adjectives (7.9%). This fits the style of COCO captions: they name things and say where they are.

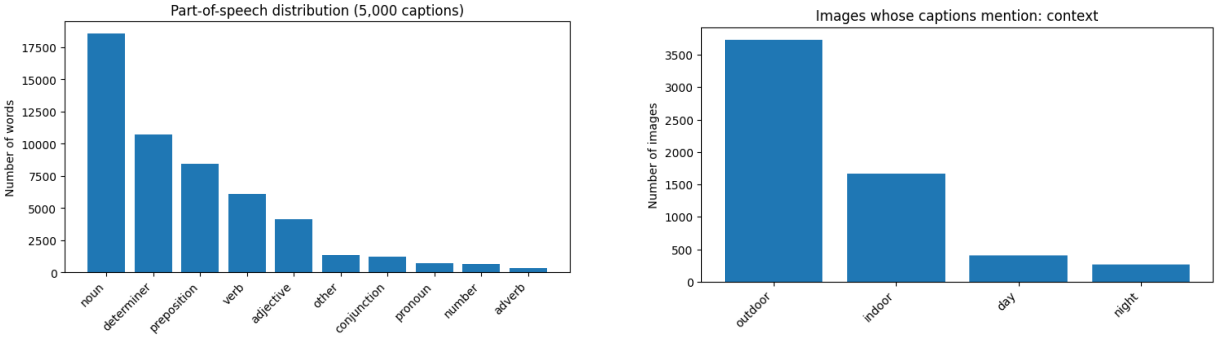


Figure 5. Parts of speech in 5,000 captions (left) and the number of images whose captions mention indoor, outdoor, day or night words (right).

To measure diversity we looked for colour words, shape and size words, indoor and outdoor words, day and night words, and spatial words (for example next to, behind, under) in the captions. Outdoor words appear in 3,737 images and indoor words in 1,663. Day and night are rarely mentioned (404 and 267 images). COCO does not give these attributes, so they come from the caption words and describe what the annotators chose to write. The notebook also draws the top 50 object classes, the number of objects per image, a co-occurrence matrix of the 20 most frequent classes, and a scatter plot of scene complexity against caption length. All plots are saved in the Drive folder.

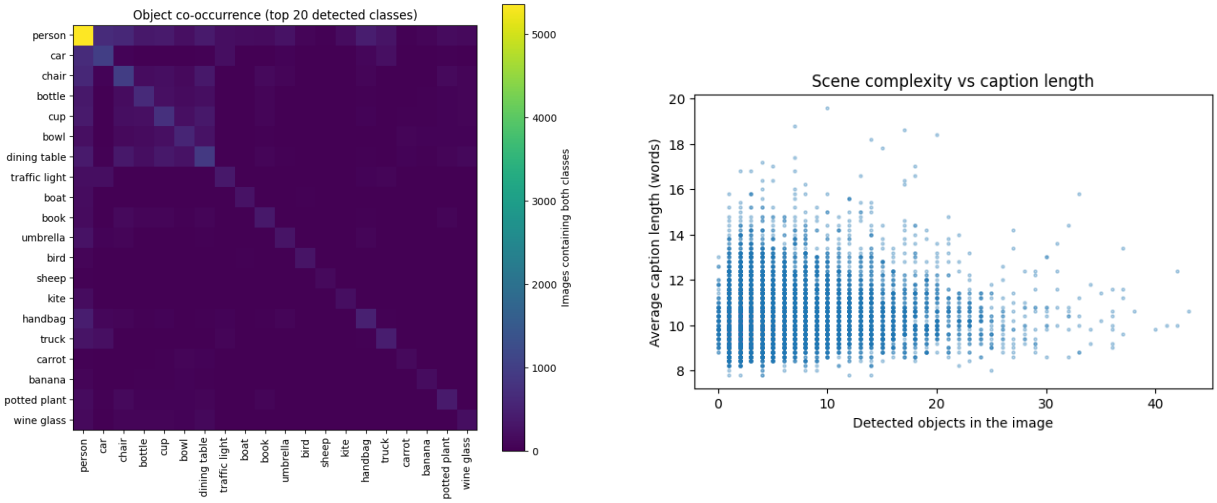


Figure 6. Which classes appear together (left) and scene complexity against caption length (right). Person appears together with almost everything; crowded scenes do not get clearly longer captions.

7. Metadata

For each of the 10,000 knowledge-base images we save one record, linked to the image id. It contains the five captions, the detected objects with their scores and boxes, the number of objects of each class, the human COCO objects (kept as a reference), the scene type, the colour, shape, context and spatial words found in the captions, and the position (row) of the image in the embedding file. For validation and test images we save two separate files. One has the objects detected in the image; retrieval is allowed to use it. The other has the human COCO objects; it is only for evaluation, for example to measure hallucinations later, and must never be used for retrieval. The caption-based attributes exist only for knowledge-base images, because using the captions of test images would be leakage.

8. Embeddings

An embedding is a list of numbers that describes an image or a sentence, so that similar things get similar numbers. We used the pretrained CLIP model (openai/clip-vit-base-patch32). CLIP puts images and text in the same space, with 512 numbers per item. This lets us compare an image with captions directly.

The function **embed_images()** loads each image (rotation fixed, RGB), lets the CLIP processor prepare it and returns its vector; images go through in batches of 64. **embed_texts()** does the same for the cleaned captions in batches of 256. The function **to_unit_vectors()** then scales every vector to length 1. With vectors of length 1, the inner product of two vectors equals their cosine similarity, the usual measure for CLIP. The images are never shuffled, so row k of a vector file always belongs to image number k of the id list.

We created 10,000 knowledge-base image vectors, 50,000 caption vectors, 100 validation vectors, 100 Test A vectors and 1,000 Test B vectors. **check_vectors()** verifies that each file has the right shape, contains no missing numbers and that every vector has length 1. **cached_vectors()** loads a file from Drive if it already exists and computes it only when it does not, so a second run takes seconds. We also checked that the vectors of the 100 Test A images are the same inside the Test B file.

9. Knowledge base and FAISS index

FAISS is a library for fast similarity search. We built two indexes with build_index(): one for the 10,000 image vectors (20.48 MB, built in 0.065 seconds) and one for the 50,000 caption vectors (102.4 MB, 0.115 seconds). Both are IndexFlatIP indexes. "Flat" means the query is compared with every stored vector, so the answer is exact. With this size the search is still very fast, so no approximation is needed. "IP" means inner product, which is cosine similarity for our vectors. Each vector is linked to its image id, captions and metadata by its row number.

Retrieval example. We used a test image of a person kiteboarding as a query. The five most similar knowledge-base captions were: “A man kiteboarding on top of the ocean”, “A man kiteboarding over the ocean next to shore”, “A parasailer holding on to the sail and waiting to land”, “A person kiteboarding over the ocean on waves” and “Someone in a wetsuit in the blue waters parasailing”. The results are close to the image, but two of them describe parasailing, which is similar but not the same. This shows that retrieval finds useful evidence that is not always exact. Handling this is the job of the RAG techniques in the next phases.



Figure 7. A test image (left) and its closest knowledge-base images.

10. Quality control and leakage prevention

Quality control is built into the code with checks (called assertions) that stop the program if something is wrong. The checks cover the number of images in each set, unique ids, five non-empty captions per image, missing, corrupted and duplicate images, shared ids and shared pictures between sets, the shape of the embeddings, missing numbers, vector length, and whether the original captions were left unchanged. All of them passed.

For leakage, three rules protect the experiment. First, validation and test images are different from the knowledge-base images, by id and by fingerprint. Second, reference captions of validation and test images are never placed in the knowledge base or the indexes. Third, a test image can never be retrieved for itself. We tested this by searching the image index with the 100 validation images and the 1,000 Test B images (Test A is inside Test B), 1,100 queries in total, taking the 10 closest results each time. None of the results was a query image.

11. The database

Everything is saved in the Drive folder RAG-Phase1:

- **images/**: the images of the knowledge base, the validation set and the test set (the test folder holds all 1,000 images; the first 100 ids are Test A).
- **splits.json and captions.json**: the ids of the four sets and the five original captions of each image.

- **metadata/**: kb_records.json (one record per knowledge-base image), kb_caption_rows.json (50,000 rows with raw and cleaned caption), query_objects.json, eval_objects.json, the detector results and settings, statistics.json, and the audit files (cleaning report, fingerprints, leakage audit).
- **embeddings/**: the vector files (.npy) and their id files.
- **index/**: the two FAISS index files.
- **plots/**: all graphs, before and after preprocessing.
- **phase1_documentation.json**: the settings of the whole phase in one file.

Drive folder: RAG-Phase1

12. Conclusion

Phase 1 made an tidy set of data. This set contains 10,000 images from the knowledge base and 50,000 captions, plus 100 separate images for validation and two test sets with 100 images and 1,000 images. We checked all sets for overlap. We also pulled out objects from the images examined the set before and after cleaning and made CLIP embeddings, for both images and captions. We built two FAISS indexes for search. Automatic checks proved the test data stays inside its set. Now every group has a validated base to build and compare their RAG techniques in Phase 2.